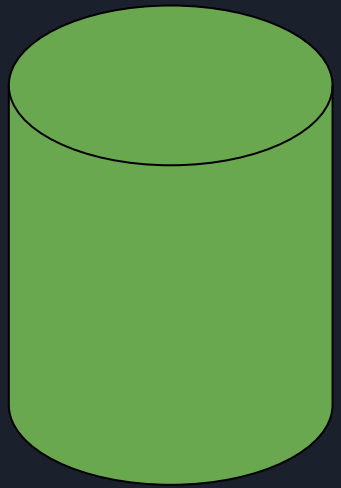
GESTIÓN DE DATOS

PL/SQL - FUNCIONES - CONDICIONALES

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Introducción a PL/SQL
- Aclaración: pl/sql VS sql/psm VS pl/pgsql
- Definición de "función"
  - Partes de una función
  - Palabras clave
- Manejo de condicionales

DEFINICIÓN: PL/SQL



¿Porqué PL/SQL?



SQL: Ya sabemos que es

Pero…

- No toma decisiones
- No maneja estados
- No tiene flujos

Para esto existe PL/SQL

¿pl/sql? ¿sql/psm? ¿pl/pgsql?



Vamos a hacer esto fácil:

- En Oracle se llama PL/SQL
- En MariaDB se llama SQL/PSM
- En PostgreSQL se llama PL/pgSQL

En la práctica y a nivel general, se habla de PL/SQL

DEFINICIÓN: PL/SQL

PL/SQL = Procedural Language / Structured Query Language

Lenguaje de programación basado en SQL, soporta parámetros, variables, estructuras de control (condicionales, ciclos) y manejo de excepciones

Soportado por varios dbms relacionales (oracle, sqlserver, mysql, mariadb, postgresql…)

DEFINICIÓN: FUNCIÓN



DEFINICIÓN: FUNCIÓN

Bloque de código en algún lenguaje de programación que puede realizar operaciones de cálculo, lectura o escritura.

Puede tener o no parámetros, y puede retornar un valor, un objeto o no retornar nada (void).

DEFINICIÓN: Procedimiento



Entonces... ¿Qué es procedimiento?

Un procedimiento almacenado (o simplemente "procedimiento"), al igual que una función, es un conjunto de instrucciones y lógica en pl/sql.

Se usan más que todo para operaciones de escritura, o para operaciones de lectura que retornen un set de datos grande que posteriormente será recorrido por un componente en la capa de aplicación o de lógica.

ALGUNAS diferencias...

Función	Procedimiento
Siempre retorna un valor	"Hace algo" pero no retorna un valor
0 o más parámetros de entrada	0 o más parámetros de entrada y/o salida
Se recomienda (pero no está limitado a) su uso para operaciones de lectura	Se usa para operaciones de lectura y/o escritura
Se puede ejecutar en una sentencia sql tipo dml	No se puede ejecutar directamente desde una sentencia sql tipo dml sino mediante el comando CALL

ALGUNAS diferencias...

=> Si necesitas "return", es una función

=> Si necesitas alterar datos (insert, update, delete), es un procedimiento

DEFINICIÓN: Deterministica



DEFINICIÓN: Deterministica

Una función determinística es una función que siempre devolverá el mismo resultado, sin importar cuantas veces se ejecute o bajo que condiciones. El resultado es completamente predecible

Una función no determinística, por el contrario, es aquella cuyo valor de retorno no puede predecirse, o puede cambiar en cada ejecución

Ejemplos ¿?¿?¿?

Mencionar ejemplos de funciones determinísticas y de no determinísticas

CONCEPTO: Función pl/sql



CONCEPTO: Función pl/sql

Una función en pl/sql se asemeja mucho a lo que es en cualquier otro lenguaje de programación.

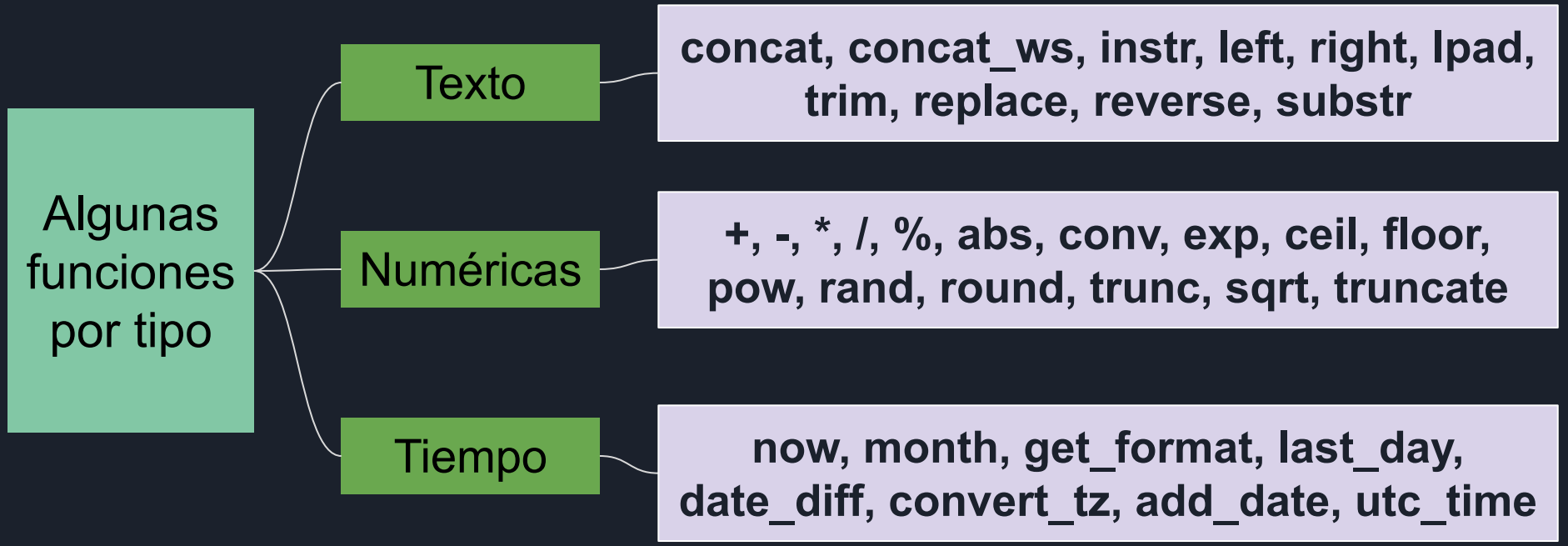
En mysql, una función es una secuencia de pasos que realiza operaciones para retornar un valor, y generalmente no modifica datos en la base de datos (es posible que lo haga pero no es lo usual).

Funciones en mariadb/mysql

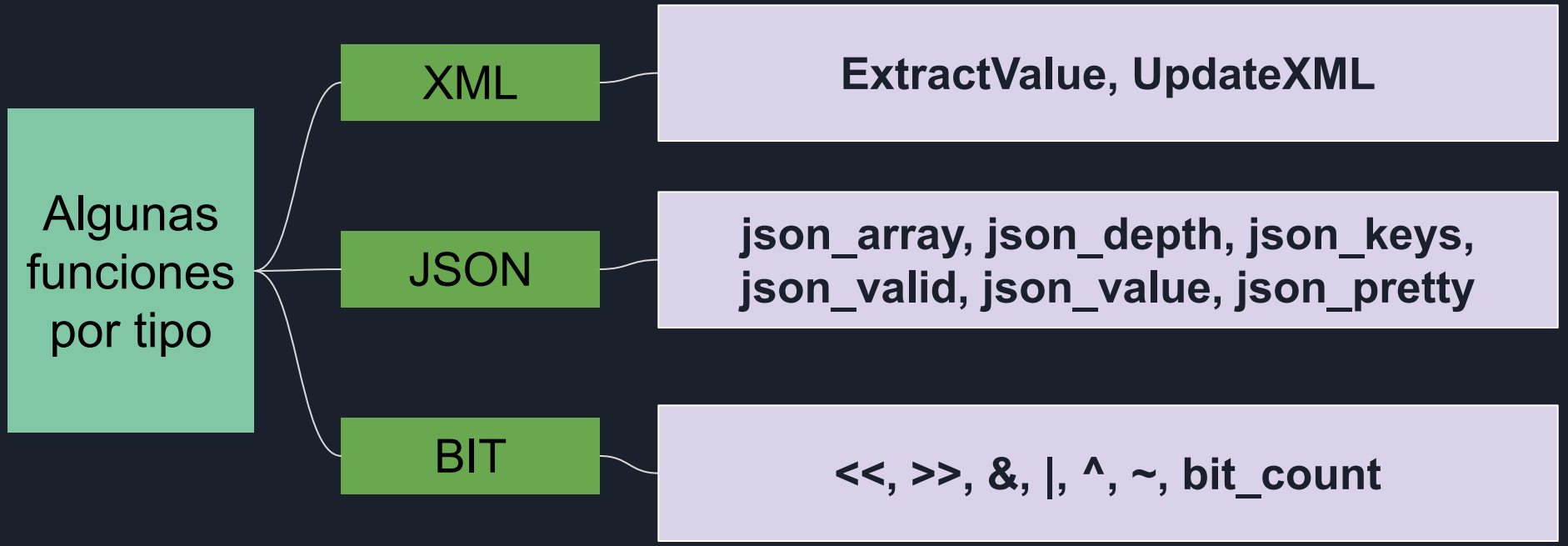
Consejo muy personal:

"Las funciones en pl/sql deberían usarse sólo para operaciones de lectura"

Funciones nativas en mariadb/mysql



Funciones nativas en mariadb/mysql



Estructura de una función

Función

- Encabezado
  - También llamado "definición".
    - Nombre de la función (obligatorio)
    - Parámetros de entrada (0 ó mas)
    - Tipo de dato de retorno (obligatorio)
- Cuerpo
  - Es la funcionalidad o la lógica como tal. Qué es lo que hará la función para calcular lo que va a retornar
- Documentación
  - Realmente no hace parte de la estructura. Pero… si ayuda a entender para qué sirve!

Ejemplo: función nativa instr

Nombre de la función	instr
Parámetro(s) de entrada	texto original, texto a buscar
Tipo de dato de retorno	int
Descripción	Busca la posición del "texto a buscar" dentro de la cadena "texto original"
Ejemplo	instr('hola mundo', 'a') ⇒ Retorna 4

¿Para qué crear funciones?

- Funcionalidad específica para cierto modelo relacional
- Funcionalidad que no existe con las funciones nativas
- Exponer a la capa de aplicación lógica que es propia de bases de datos
- Abstraer la complejidad de una operación (o serie de operaciones)

¿Como crear funciones en mariadb?

Encabezado

Cuerpo

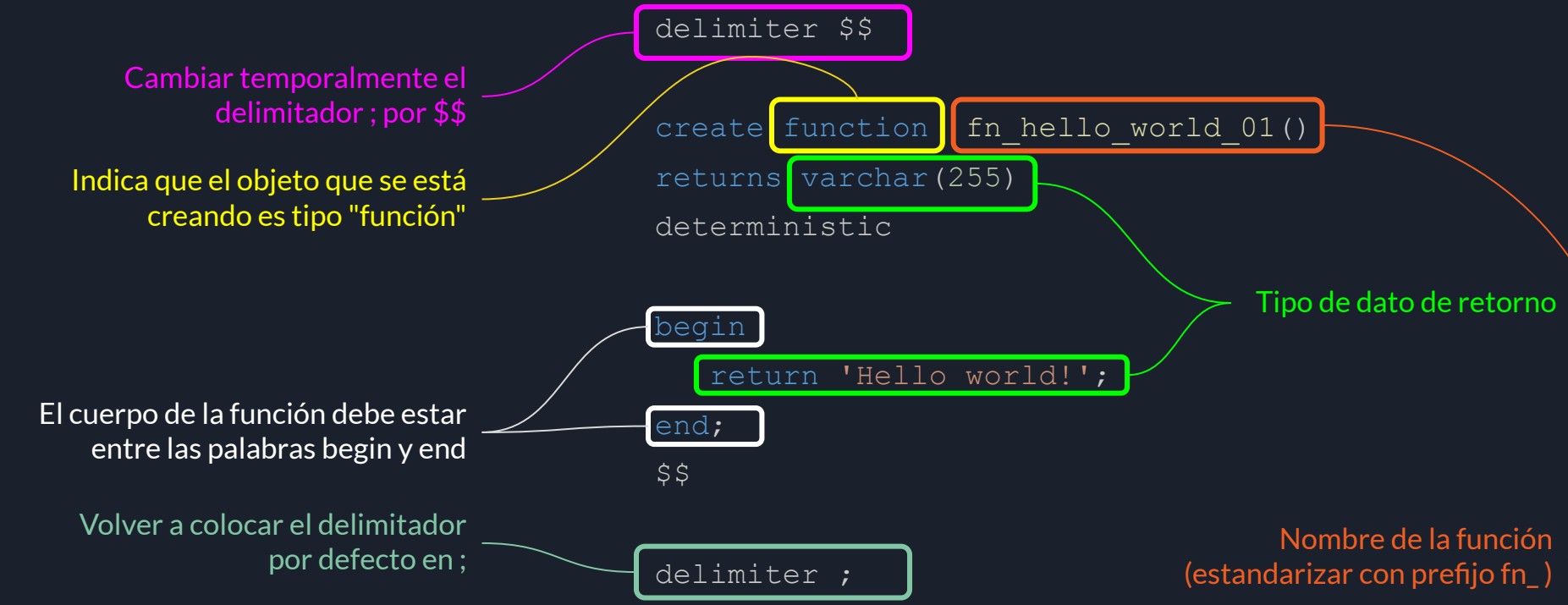
```
delimiter $$

create function fn_hello_world_01()
returns varchar(255)
deterministic

begin
   return 'Hello world!';
end;
$$

delimiter ;
```

¿Como crear funciones en mariadb?



Función con parámetros

Encabezado

Cuerpo

```
delimiter $$

create function fn_sum_two_integer_numbers(
    p_number_1 int,
    p_number_2 int
)
returns int
deterministic
begin
    declare v_result int;

    set v_result := p_number_1 + p_number_2;

    return v_result;
end;
$$
```

Función con parámetros

Nombre de los parámetros (estandarizar con prefijo p_)

Nombre de variable (estandarizar con prefijo v_)

Palabra clave para declarar variables nuevas

Palabra clave para asignar valor a una variable

Operador para asignar un valor a una variable

```
delimiter $$

create function fn_sum_two_integer_numbers(
    p_number_1 int,
    p_number_2  int
)
returns int
deterministic
begin
    declare v_result int;

    set v_result := p_number_1 + p_number_2;

    return v_result;
end;
$$
```

Condicionales: if-then-elseif-else

```
(...)
begin
   declare v_result varchar(10);

   if p_color = 'BLACK' then
       set v_result := 'NEGRO';
   elseif p_color = 'WHITE' then
       set v_result := 'BLANCO';
   elseif p_color = 'YELLOW' then
       set v_result := 'AMARILLO';
   else
       set v_result := 'N/A';
   end if;

   return v_result;
end;
(...)
```

Condicionales: if-then-elseif-else

Todo if ***debe*** cerrarse con end if

Los elseif y else son opcionales y dependen únicamente de la lógica

```
(...)
begin
   declare v_result varchar(10);

   if p_color = 'BLACK' then
      set v_result := 'NEGRO';
   elseif p_color = 'WHITE' then
      set v_result := 'BLANCO';
   elseif p_color = 'YELLOW' then
      set v_result := 'AMARILLO';
   else
      set v_result := 'N/A';
   end if;

   return v_result;
end;
(...)
```

Condicionales: case-when-else

```
(...)
begin
   declare v_result varchar(10);

   case p_color
   when 'BLACK' then
       set v_result := 'NEGRO';
   when  'WHITE' then
       set v_result := 'BLANCO';
   when  'YELLOW' then
       set v_result := 'AMARILLO';
   else
       set v_result := 'N/A';
   end case;

   return v_result
end
(...)
```

Condicionales: case-when-else

Todo case ***debe*** cerrarse con end case

Debe existir al menos un when. El else es opcional. Esto depende de la lógica de cada función

```
(...)
begin
   declare v_result varchar(10);

   case p_color
   when 'BLACK' then
       set v_result := 'NEGRO';
   when 'WHITE' then
       set v_result := 'BLANCO';
   when 'YELLOW' then
       set v_result := 'AMARILLO';
   else
       set v_result := 'N/A';
   end case;

   return v_result
end
(...)
```

Llamar a una función desde otra

```
(...)

    declare v_hello varchar(30);
    declare v_result varchar(30);

    set v_hello := 'hello world!';
    set v_result := substr(v_hello, 1, 5); // hello
    set v_result := substr(v_hello, 7);    // world!
    set v_result := substr(v_hello, -6);   // world!
    set v_result := substr(v_hello, -6);   // wor


(...)
```

Llamar a una función desde otra

Se coloca el nombre de la función y los parámetros.

Sirve tanto para funciones nativas como para funciones creadas por el usuario

```
(...)

    declare v_hello varchar(30);
    declare v_result varchar(30);

    set v_hello := 'hello world!';
    set v_result := substr(v_hello, 1, 5); // hello
    set v_result := substr(v_hello, 7);    // world!
    set v_result := substr(v_hello, -6);   // world!
    set v_result := substr(v_hello, -6);   // wor

(...)
```

¿Cómo probar una función?

```
select fn_my_function();

select fn_my_function(x);

select fn_my_function(x, y);
```

¿Cuando IF y cuando CASE?

Aunque es una decisión muy personal, una guía es esta:

Usar CASE cuando:

- La condición es la misma y las acciones sean distintas (ej: lista de colores)

Usar IF cuando:

- Lógica compleja (ej: nombre like '%lina%')
- Rangos (ej: x between 1 and 10)
- Condiciones múltiples

RESUMEN HASTA AHORA

delimiter	Modificar el delimitador por defecto (;)
declare	Crear variables con su tipo de dato
set	Asignar valor a variable
create function	Crear una función
return	Indicar a la función que debe retornar un valor
returns	Especifica que tipo de dato debe retornar la función
deterministic	Indica que la función es determinística
if / elseif / else / end if	Condicionales
case / when / else / end case	Condicionales

EJERCICIOS (1/5)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_addition_01
- Parámetros: valor inferior, valor_superior (p_min, p_max)
- Tipo de dato de retorno: int
- Descripción: Realizar la suma de p_min y p_max
- Ejemplo: Para los valores de entrada 1 y 10 el cálculo sería 1+10 = 11

EJERCICIOS (2/5)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_triangle_area
- Parámetros: lado, altura (p_side, p_height)
- Tipo de dato de retorno: real
- Descripción: Calcular el área de un triángulo, la cual se hace con la fórmula (base * altura) / 2
- Ejemplo: Para los valores de entrada 3 y 5, el valor de retorno es 7.5

EJERCICIOS (3/5)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_arithmetic_operation_01
- Parámetros: valor 1, valor 2, operación (p_val1, p_val2, p_operation)
- Tipo de dato de retorno: real
- Descripción: Realizar la operación especificada en "p_operation" para los números p_val1 y p_val2. Los posibles valores para p_operation son: "addition", "subtraction", "multiplication", "division". Si se especifica cualquier otro valor entonces la función debe retornar 0.
- Ejemplo: Para los valores de entrada 1, 10, "addition" debe retornar 11. Para los valores 10, 5, "substraction" debe retornar 5.

EJERCICIOS (4 / 5)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_number_type_01
- Parámetros: número (p_number)
- Tipo de dato de retorno: varchar
- Descripción: Si el número de entrada es par debe retornar "EVEN", en caso contrario debe retornar "ODD".
- Ejemplo: Para el valor 2 debe retornar EVEN. Para el valor 3, ODD.

EJERCICIOS (5/5)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_is_triangle_area_even_01
- Parámetros: número (p_side, p_height)
- Tipo de dato de retorno: varchar
- Descripción: Si el área del triángulo es un número par, debe retornar "YES". En caso contrario, debe retornar "NO". Debe apoyarse en las funciones de los ejercicios 1 y 3.
- Ejemplo: Para los valores de entrada 3 y 5, el valor de retorno es NO

¿PREGUNTAS?

